

CSE 3131

Compiler Design

Md. Emdadul Haque

Lecturer

Department of Computer Science & Engineering
North Bengal International University

Error Handling

The tasks of the Error Handling process are **to detect each error, report it to the user, and then make some recovery strategy** and implement them to handle the error. During this whole process processing time of the program should not be slow.

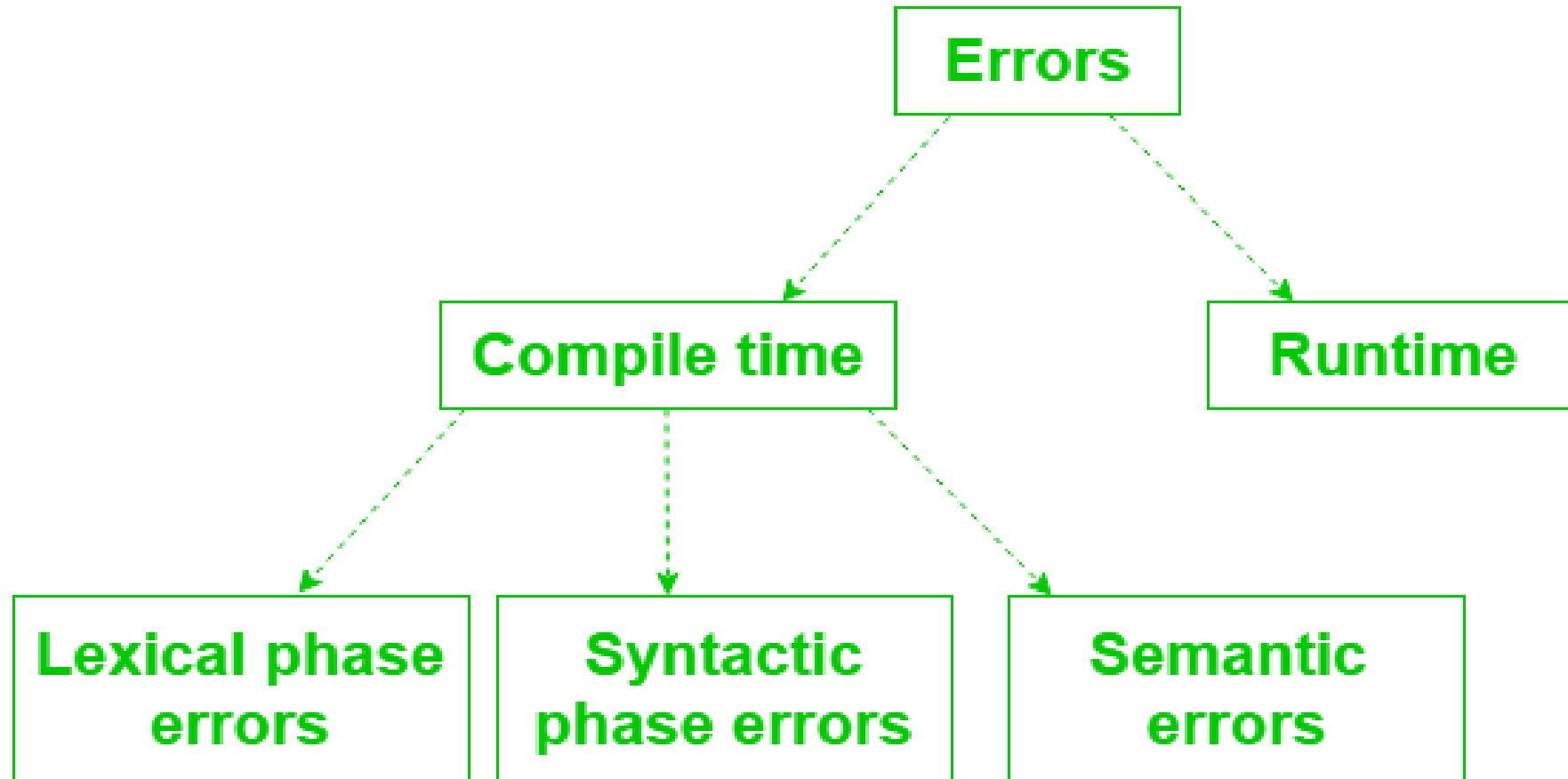
Functions of Error Handler:

- Error Detection
- Error Report
- Error Recovery

Error handler=Error Detection + Error Report + Error Recovery.

What is error handler? What does it perform?

Types of Errors



Compile Time Vs Run Time Errors

Key	Compile Time Errors	Runtime Errors
Reference	Compile-time errors are generally referred to the error corresponding to syntax or semantics.	Runtime errors on the other hand refer to the error encountered during the execution of code at runtime.
Detection	Compile-time errors get detected by compiler at the time of code development.	Runtime time errors are not get detected by compiler and hence identified at the time of code execution.
Fixation	Compile-time errors as already mentioned can get fixed at the time of code development.	Runtime time errors are getting to fixing state after once code get executed and errors get identified.

A compile-time error generally refers to the errors that correspond to the semantics or syntax. A runtime error refers to the error that we encounter during the code execution during runtime. We can easily fix a compile time error during the development of code. A compiler cannot identify a runtime error.

Difference between compile time and run time error?

Lexical Error

During the lexical analysis phase this type of error can be detected.

Lexical error is a sequence of characters that does not match the pattern of any token. Lexical phase error is found during the execution of the program.

Lexical phase error can be:

- Spelling error.
- Exceeding length of identifier or numeric constants.
- Appearance of illegal characters.
- To remove the character that should be present.
- To replace a character with an incorrect character.
- Transposition of two characters.

What types of error occurred in lexical phase? .

Lexical Error (Contd.)

```
Void main()
{
    int x=10, y=20;
    char * a;
    a = &x;
    x= 1xab;
}
```

In this code, 1xab is neither a number nor an identifier. So this code will show the lexical error.

What types of error occurred in lexical phase? .

Error recovery for lexical errors

Panic Mode Recovery

- In this method, successive characters from the input are removed one at a time until a designated set of synchronizing tokens is found. Synchronizing tokens are delimiters such as ; or }
- The advantage is that it is easy to implement and guarantees not to go into an infinite loop
- The disadvantage is that a considerable amount of input is skipped without checking it for additional errors

Example: int a, ~~5abcd~~, sum, ~~\$~~2;

What are the recovery strategies to solve the lexical errors?.

Syntax Error

During the syntax analysis phase, this type of error appears. Syntax error is found during the execution of the program.

Some syntax error can be:

- Error in structure
- Missing operators
- Unbalanced parenthesis

When an invalid calculation enters into a calculator then a syntax error can also occurs. This can be caused by entering several decimal points in one number or by opening brackets without closing them.

What types of error occurred in syntax analysis phase? .

Syntax Error (Contd.)

- For example 1: Using "=" when "==" is needed.

```
16 if (number=200)
17     count << "number is equal to 20";
18 else
19     count << "number is not equal to 200"
```

In this code, if expression used the equal sign which is actually an assignment operator not the relational operator which tests for equality.

Due to the assignment operator, number is set to 200 and the expression number=200 are always true because the expression's value is actually 200.

What types of error occurred in syntax analysis phase? .

Syntax Error (Contd.)

Example 2: Missing semicolon:

- `int a = 5        // semicolon is missing`
- `int a = 5;`

Example 3: Errors in expressions:

- `x = (3 + 5;    // missing closing parenthesis ')'`
- `y = 3 + * 5;    // missing argument between '+' and '*'`

What types of error occurred in syntax analysis phase? .

Error recovery for Syntax Errors (Contd.)

1. Panic Mode Recovery

- In this method, successive characters from the input are removed one at a time until a designated set of synchronizing tokens is found. Synchronizing tokens are delimiters such as ; or }
- The advantage is that it's easy to implement and guarantees not to go into an infinite loop
- The disadvantage is that a considerable amount of input is skipped without checking it for additional errors

What are the recovery strategies to solve the syntax errors?.

Error recovery for Syntax Errors (Contd.)

2. Error production

- If a user has knowledge of common errors that can be encountered then, these errors can be incorporated by augmenting the grammar with error productions that generate erroneous constructs.
- If this is used then, during parsing appropriate error messages can be generated and parsing can be continued.
- The disadvantage is that it's difficult to maintain.

Grammar:

$S \rightarrow A$

$A \rightarrow aA | bA | a | b$

$B \rightarrow cd$

New Grammar:

$E \rightarrow SB$

$S \rightarrow A$

$A \rightarrow aA | bA | a | b$

$B \rightarrow cd$

string is : abcd

What are the recovery strategies to solve the syntax errors?.

Error recovery for Syntax Errors (Contd.)

3. Phase level recovery

When an error is discovered, the parser performs local correction on the remaining input.

You can correct the error by deleting extra semicolons, replacing commas with semicolons, or reintroducing missing semicolons. To prevent going in an infinite loop during the correction. In this way, the parser can continue to operate on its execution.

Int a, b, → int a, b;

What are the recovery strategies to solve the syntax errors?.

Error recovery for Syntax Errors (Contd.)

4. Global Correction

- The parser examines the whole program and tries to find out the closest match for it which is error-free.
- The closest match program has less number of insertions, deletions, and changes of tokens to recover from erroneous input.
- Due to high time and space complexity, this method is not implemented practically.

What are the recovery strategies to solve the syntax errors?.

Semantic Error

During the semantic analysis phase, this type of error appears. These types of error are detected at compile time.

Most of the compile time errors are scope and declaration error.

For example: undeclared or multiple declared identifiers. Type mismatched is another compile time error.

The semantic error can arise using the wrong variable or using wrong operator or doing operation in wrong order.

Some semantic error can be:

- Incompatible types of operands
- Undeclared variable
- Not matching of actual argument with formal argument

What types of error occurred in semantic phase? .

Semantic Error (Contd.)

- **Example 1:** Use of a non-initialized variable:

```
int i;  
void f (int m)  
{  
    m=t;  
}
```

In this code, t is undeclared that's why it shows the semantic error.

- **Example 2:** Type incompatibility:

```
int a = "hello";    // the types String and int are not compatible
```

- **Example 3:** Errors in expressions:

```
String s = "...";
```

```
int a = 5 - s;    // the - operator does not support arguments of type String
```

What types of error occurred in semantic phase? .

Error recovery for Semantic errors

- If the error “**Undeclared Identifier**” is encountered then, to recover from this a symbol table entry for the corresponding identifier is made.
- If data types of two operands are incompatible then, automatic type conversion is done by the compiler.

What types of error occurred in semantic phase? .

Why Error Handling is Important?

Error handling is important because it makes it easier for the end users of your code to use it correctly. Another important issue is that it makes your code easier to maintain. Error handling makes it easier to embed input specifications into the code, so you don't have to look up the design when you write and later maintain the code.

THANK YOU ALL